Stage 2 Implementation Report

AI-powered Study Planner | EECS 3311 | Rafat Kabir
Progress draft: F01–F09 and UC-11, November 9, 2026.

1. Overview and status

Students can create courses, upload materials, generate summaries, extract concepts, ask questions about their materials, generate personalized study schedules, study AI-generated flashcards, and generate, attempt, and grade quizzes through a Vaadin GUI or Java CLI. Spring Boot stores courses, materials, summaries, concept sets, chat history, profiles, plans, flashcard decks, quizzes, attempts, grade results, and calendar events in H2. F03 uses local Ollama; the HTTP adapter also supports hosted OpenAI-compatible providers.

F01–F09 and UC-11 are implemented. F10 has event storage and plan synchronization; its full interface remains pending. Course editing/deletion remain pending. Authentication supports one locally configured account and does not count as a project feature.

2. Feature-to-implementation mapping

F01: Course Creation. Deterministic. References: Stage 1 Task 1.2 F01, UC-01, SD01, and Task 4 F01. Students enter a name, description, start date, and end date. The system validates the input, saves an owned course, and displays it on the dashboard. Video: pending.

Part	Files, responsibilities, and key methods
GUI	`frontend/.../views/MainView.java`: `showSignIn`, `showDashboard`, `openCourseForm`, `refreshCourses`, and `renderCourses` handle sign-in, forms, errors, and course cards. `themes/study-planner/styles.css` styles the UI.
API client	`frontend/.../course/CourseApi.java`: `createCourse`/`viewCourses` call the backend. `CourseApiFactory.connect` creates a client for each signed-in view. Frontend `Course` and `CreateCourseRequest` records carry API data.
REST API	`backend/.../course/CourseController.java`: `createCourse`, `viewCourses`, and `getCourse` expose POST/GET `/api/courses` and GET `/api/courses/{id}`.
Application and domain	`CourseService` coordinates creation and retrieval. `User.createCourse`/`viewCourses` create and retrieve owned courses. Backend `Course` validates domain data; `CreateCourseRequest` validates API fields.
Storage	`CourseRepository` defines storage operations. `JdbcCourseRepository.save`/`findByOwner`/`findByIdAndOwner` use SQL. `schema.sql` defines the table. H2 files persist under `backend/data/`.
Authentication and errors	`SecurityConfiguration` verifies credentials; `CsrfController.token` supports protected writes. `ApiExceptionHandler` handles invalid requests and storage failures. The authenticated student determines course ownership.
CLI	`cli/StudyPlannerCli.java`: `main`/`run` provide `create`, `list`, and `help` using the same authenticated API.

Flow: form or CLI → CourseController → CourseService → User.createCourse → CourseRepository → H2. The GUI then reloads the saved course list. Failed submissions show errors and preserve form input.

All four fields are required. Names allow 120 characters; descriptions allow 2000. Text is trimmed, and end dates must be on or after start dates. Same-day courses and duplicate names are allowed. Course queries are restricted to their owner.

F02: Upload Learning Materials. References: Stage 1 Task 1.2 F02, UC-02, SD01, and Task 4 F02. Open an owned course, select files, and view the saved materials and extracted text. Upload/validation/text extraction are deterministic; the planned AI processing belongs to later features. Video: pending.

Part	Implementation and key methods
GUI / client	`MainView.openCourse` opens `CourseMaterialsPanel`; upload, `refreshMaterials`, and `viewText` show results and errors. Each file offers View text and Download original. `CourseApi.uploadMaterial/listMaterials/materialContent/downloadMaterial` call the authenticated API.
API / service	`CourseController.uploadMaterial/listMaterials/materialContent/downloadMaterial` delegate to `MaterialService`, which verifies course ownership before access.
Domain / parsing	`Course.addMaterial` creates `LearningMaterial`; `getContent` exposes extracted text. `MaterialParser.parse` checks filenames and parses PDF, DOCX, or UTF-8 TXT using PDFBox, POI, or UTF-8 decoding.
Storage / CLI	`MaterialRepository` and `JdbcMaterialRepository` save metadata, original BLOB, and extracted text together. `StudyPlannerCli` adds `upload`, `materials`, and `content`.

Rules: up to 10 files, 10 MiB each; unsupported, empty, corrupt, encrypted, or text-free files are rejected. No OCR is provided. Text is limited to 1,000,000 characters, PDF to 500 pages, and expanded DOCX to 20 MiB/1000 entries. API batches roll back on any failure; GUI files upload independently. Originals and text survive restarts.

F03: Document Summarization. AI-based. References: Stage 1 F03, UC-03, SD02 summary branch, and Task 4 F03. Select Summarize document; the agent summarizes source sections, synthesizes them when needed, resolves selected source-excerpt IDs to verified quotations, and saves the result. View saved summary reuses it; regeneration explicitly requests a replacement. CLI: `summarize`, `summary`, `regenerate-summary`. Video pending.

Part	Implementation
GUI / API	`frontend/.../views/SummaryActions` runs background requests; `CourseApi.summarize/viewSummary` call `backend/.../resource/SummaryController`. Progress and safe errors remain visible while inference runs.
Agent / resource	`Course.generateStudyResource` calls `StudyAssistantFacade.summarize`. `ContentAgent.summarize` plans sections and validates responses from `AIEngine.generate`. `StudyResourceFactory.createResource` constructs `Summary`, extending `StudyResource`; `view` returns its text.
Storage	`SummaryService` verifies ownership, guards concurrent generation, and checks the source hash. `SummaryRepository/JdbcSummaryRepository` save the latest summary, evidence, model, and usage. Failed regeneration preserves the previous result.

F03 accepts 40–32,000 source characters. Generation has schema-constrained output, bounded calls/time, and one repair attempt for invalid results. Missing setup, provider failures, insufficient text, invalid output, and timeouts return errors without saving. Quotations must match the source; factual accuracy still needs human review.

F04: Concept Extraction. Implemented, AI-based. References: Stage 1 F04, UC-04, SD02 concept branch, Task 4 F04, and original UML. Select Extract Concepts for an uploaded document or uploaded section; a list shows important topic names, explanations, and supporting source quotations. View saved concepts and Regenerate concepts provide reuse and replacement. Video pending.

Part	Implementation and collaboration
GUI / API / CLI	`ConceptActions` runs background requests via `CourseApi.extractConcepts/viewConcepts`. `ConceptSetController` exposes GET/POST `/api/courses/{courseId}/materials/{materialId}/concepts`. CLI: `extract-concepts`, `concepts`, `regenerate-concepts`.
Agent / patterns	`Course.generateStudyResource` → `StudyAssistantFacade.extractConcepts` → `ContentAgent.extractConcepts` → singleton `AIEngine.generate`. The agent reads `LearningMaterial.getContent`, plans sections, extracts concepts, and consolidates multi-section results. `StudyResourceFactory.createResource` creates `ConceptSet`, the second concrete `StudyResource` product.

Validation / storage	Structured output requires 1–10 concepts with names, explanations, and valid source-excerpt IDs. Concept names may paraphrase the source; readable source sentences and long-sentence tails are available as evidence, with dense lists grouped into contiguous windows. Normalized duplicate names are merged; invalid output gets one repair attempt. `ConceptSetService` checks ownership, source hashes, and concurrent requests. `ConceptSetRepository/JdbcConceptSetRepository` atomically save the latest set in H2; failed regeneration preserves it.

F04 shares F03's 40–32,000-character, 14-call, 180-second, and provider limits. It operates on a selected uploaded material, including a separately uploaded section; course-section management and whole-course extraction are not included. Exact quotations do not guarantee explanation accuracy or semantic uniqueness.

F05: Question Answering Over Materials. Implemented, AI-based. References: Stage 1 F05, UC-05, SD02 question branch, Task 4 F05, and original UML. Open a course, enter a question in Course chat, and send it. The agent finds relevant course passages, asks Ollama for an answer, checks its source references, and saves the exchange. Expand a source to read its quotation or open the document text. Video pending.

Part	Implementation and key methods
GUI / API / CLI	`CourseChatPanel` shows progress, saved messages, wrapped answers, source quotations, and errors through `CourseApi.askQuestion/chatHistory`. `ChatController` exposes POST/GET `/api/courses/{courseId}/chat`. CLI: `ask` and `history`.
Domain / AI	`Course.askQuestion` → `CourseChat.askQuestion` → `StudyAssistantFacade.answerQuestion` → `ContentAgent.answerQuestion` → `AIEngine.generate`. `CourseContextRetriever.retrieve` ranks passages using words in the question and recent conversation. The model receives up to six matching passages. Follow-up questions can also use short excerpts from two recent exchanges; new topics start without earlier answers. F05 reuses Facade and Singleton.
Storage / validation	`ChatService` checks ownership, question length, and concurrent requests. `ChatRepository/JdbcChatRepository` save complete `ChatTurn` records in H2. `CourseChat.viewHistory` returns saved exchanges. Citations include material ID, filename, text offsets, and the original quotation. `ChatAnswerValidator.isUnfinished` detects dangling introductions and empty list items. History loads in pages of 50, with older messages available.

Questions allow 1–1000 characters. Chat searches up to 50 materials and 1,000,000 extracted characters per course; larger courses get an error. Invalid or visibly unfinished AI output gets one correction attempt. Introductions without the promised content and empty list items are not saved. Failed requests preserve the question in the GUI and leave history unchanged. Insufficient-context responses are saved without citations. Broad questions such as "Tell me about this course" use passages from across the uploaded materials. For specific questions with no matching passages, the system declines without a model call; the GUI labels this as Course search. Word matching can miss relevant passages, and valid source references do not guarantee correct answers.

F06: Personalized Study-Plan Generation; UC-11: Maintain Study Profile. Implemented, hybrid. References: Stage 1 F06, UC-06/UC-11, SD03, Task 4 F06, and original UML. Open Study plan and availability, save a profile, add busy times or quiz/exam dates, then choose plan dates and session count/duration. The AI recommends topics and activities; Java finds valid times and saves the plan and calendar sessions. Video pending.

Part	Implementation and key methods
GUI / CLI / API	`StudyPlanningPanel` edits availability, preferences, habits, and events, then displays priorities, sessions, and source quotations. `CourseApi` calls `PlanningController`. CLI adds `profile`, `save-profile`, `plan`, `generate-plan`, `events`, `add-event`, and `remove-event`.
Profile / orchestration	`StudyProfile.updateAvailability/updatePreferences` validate the shared weekly profile. `PlanningService` checks ownership, dates, material limits, and available time. `Course.generateStudyResource` calls `StudyAssistantFacade.generateStudyPlan`.
AI / scheduling / factory	`PlanningAgent.generateStudyPlan` uses source excerpts, profile data, upcoming dates, and optional assessment notes through the singleton `AIEngine.generate`. It validates topics, priorities, activities, and source IDs, with one repair attempt. `StudyScheduler.allocate` chooses all times. `StudyResourceFactory.createResource` creates `StudyPlan` and `StudySession`; `getSessions` exposes the schedule.

Storage / Observer	`PlanningRepository` stores profiles, latest plans, and events in H2. `PlanPersistence.save` attaches the `Calendar` observer and calls `StudyPlan.notifyObservers`. `Calendar.update` replaces the course's generated events. Plan and calendar changes commit together; failure preserves both previous records.

Weekly study minutes are shared across courses and must fit non-overlapping availability windows. Plans contain 1–30 sessions of 15–180 minutes over 1–56 days, within course dates and no earlier than today in the profile's time zone. Sessions avoid calendar conflicts, respect weekly budgets, and finish before the earliest upcoming quiz/exam in the selected period. Changing profile/events affects the next generated plan. Assessment results can be entered as notes; saved quiz scores and weak areas now guide the next generated plan through F09. Topic quality needs source review. Limits remain 50 materials / 1,000,000 extracted characters. Topic discovery checks up to six passages, with at most two calls per passage and a shared 180-second limit. Up to 12 specific topics are collected; there is no fixed two-topic limit.

F07: Flashcard Generation. Implemented, AI-based. References: Stage 1 F07, UC-07, SD04 flashcard branch, Task 4 F07, and original UML. Expand Flashcards in a course, choose a maximum and optional materials/topic, then generate. Click a question or Show answer to reveal the answer and sources; previous/next cards hide the next answer. Saved decks reopen without AI. Video pending.

Part	Implementation and key methods
GUI / API / CLI	`FlashcardPanel` offers material selection, a 1–20 card maximum, optional topic focus, answer reveal, navigation, and saved history. `CourseApi.generateFlashcards/flashcardDecks/flashcardDeck` call `FlashcardController`: POST/GET `/api/courses/{id}/flashcards` and GET `.../{deckId}`. CLI adds `flashcards`, `decks`, and `deck`.
Planned class flow	`Course.generateStudyResource` → `StudyAssistantFacade.generateFlashcards` → `AssessmentAgent.generateFlashcards` → `AIEngine.getInstance/generate` → `StudyResourceFactory.createResource` → `FlashcardDeck`. `LearningMaterial.getContent` supplies the source text; `FlashcardDeck.viewCards` supplies cards through the API getter. Reuses Facade, Singleton, and Simple Factory.
Validation / storage	`FlashcardService` checks ownership, selected files, size, and concurrent generation. `AssessmentAgent` selects excerpts, generates batches of up to three, validates questions/answers/source IDs, removes duplicate questions, and permits one repair per batch. `FlashcardRepository` stores complete decks in H2 `flashcard_decks`. History pages contain 25 decks; failed generation preserves previous decks.

The maximum is 1–20 cards; the model may return fewer useful cards instead of padding. Empty material selection uses all course files. Selecting separately uploaded sections limits the source; a topic focus narrows passage matching. There is no separate section-management feature. Limits: 50 materials, 1,000,000 extracted characters, six 700-character excerpts, 40 model calls, and 180 seconds. Answers must copy a sentence or short phrase from a cited excerpt (ignoring case and repeated whitespace). This rejects invented answers even when the model gives a valid source ID. Questions are AI-written; their relevance and semantic uniqueness still need review. Saved decks are snapshots.

F08: Quiz Generation. Implemented, AI-based. References: Stage 1 F08, UC-08, SD04 quiz branch, Task 4 F08, and original UML. Expand Quizzes, choose count/difficulty/topics/materials and optional types, then generate. Start a quiz, save a draft, reopen it, and finish to review expected answers and source quotations. F09 now adds automatic grades and feedback. Video pending.

Part	Implementation and key methods
GUI / client / CLI	`QuizPanel`, `QuizData`, and `CourseApi` provide generation, saved history, mixed questions, draft/resume, finish, review, progress, and errors. CLI adds `generate-quiz`, `quizzes`, `quiz`, `start-quiz`, `quiz-attempts`, `quiz-attempt`, `save-quiz`, and `finish-quiz`.
Planned class flow	`Course.generateStudyResource` → `StudyAssistantFacade.generateQuiz` → `AssessmentAgent.generateQuiz` → `QuizGenerator` → singleton `AIEngine.generate` → `StudyResourceFactory.createResource` → `Quiz`. `Quiz.startQuiz` returns the public questions used by the API. Reuses Facade, Singleton, and Simple Factory; Quiz is the fifth product.
Generation / evidence	The agent retrieves course passages and asks the model to select a missing phrase from each source fact and optional MCQ distractors. Java builds a fill-in-the-blank question, checks that its key restores the original statement, rejects repeated question stems and choices, and shuffles MCQ choices. One repair per fact; the exact question count must pass before saving.

API / storage / attempts	`QuizController/QuizService/QuizRepository` enforce ownership and store quizzes/attempts in H2. POST/GET `/api/courses/{id}/quizzes` generate/list; quiz and attempt routes read/start/save/finish. Draft responses omit keys; completed attempts permit answer review. Version checks prevent stale saves. Earlier quizzes remain unchanged on generation failure.

Inputs: 1–20 questions, EASY/MEDIUM/HARD, optional topics (500 characters), and SHORT_ANSWER/MULTIPLE_CHOICE preferences; empty types default to short answer. Selected files represent uploaded sections. Limits: 50 materials / 1,000,000 characters, six 700-character excerpts, 40 calls / 180 seconds. Unusable facts are skipped while the bounded generation budget remains. Draft answers allow 4000 characters; finishing requires all answers and freezes the attempt. Quiz history and recent attempts show 25 entries per page/list. Difficulty guides the phrase selected for the blank. Distractor plausibility, ambiguity, relevance, and semantic difficulty still need review.

F09: Automatic Quiz Grading. Implemented, hybrid. References: Stage 1 F09, UC-09, SD05, Task 4 F09, and original UML. Submit a completed attempt to receive points, percentage, question feedback, weak areas, and source-based answer review. Saved results reopen without AI; older finished attempts offer Grade saved attempt. Video pending.

Part	Implementation and key methods
Planned grading flow	`Quiz.submitQuiz` → `QuizGradingService.gradeQuiz`. For each question, `setStrategy` chooses `DeterministicGradingStrategy.grade` or `AIGradingStrategy.grade`. The AI strategy uses singleton `AIEngine.generate` when necessary. Results are combined into `GradeResult`. Strategy is the fifth planned pattern.
Criteria / scoring	Choice questions use the saved key, including binary True/False choices when present; F08 currently generates MCQs and short answers. Short answers matching the key after NFC, whitespace, case, and final sentence-punctuation normalization receive full marks without AI. Other responses receive an AI verdict: CORRECT, PARTIAL, or INCORRECT. Java maps this to full/half/zero stored points and calculates weighted totals and percentage.
API / GUI / CLI	`QuizController/QuizService` connect submission to grading. `QuizPanel` displays results and keeps inputs on failure. `QuizData` and `CourseApi` carry typed grades. Existing CLI `finish-quiz` now grades; `quiz-attempt` reads the result, and `grade-quiz` grades a previously finished attempt.
Storage / planning	`QuizSubmissionPersistence` commits the finished attempt and `GradeRepository` result together. H2 adds `quiz_grades`. Version checks and attempt locks prevent competing writes; cached results prevent repeat model calls. `PlanningAgent` reads the latest graded attempt for each of up to five recent course quizzes so saved weak areas influence the next generated plan.

All responses and grading criteria are checked before AI. Semantic grading gets one repair per question, at most 40 calls / 180 seconds. The model returns a verdict and reason code; Java builds feedback and review facts from the saved answer and source statement. Invalid output or provider/storage failures leave the saved attempt and grade unchanged; no partial grade or guessed fallback score is saved. The form retains answers for saving/retry. Completed responses and results are immutable; use a new attempt to retry. AI marks still need source review. Successful results retain model identity and reported usage; failed operations are not included in saved-result counts.

Remaining Stage 1 scope:

ID	Feature	ID	Feature
F10	Calendar — partial; full interface and Course/Quiz subjects pending		

3. Design patterns and diagrams

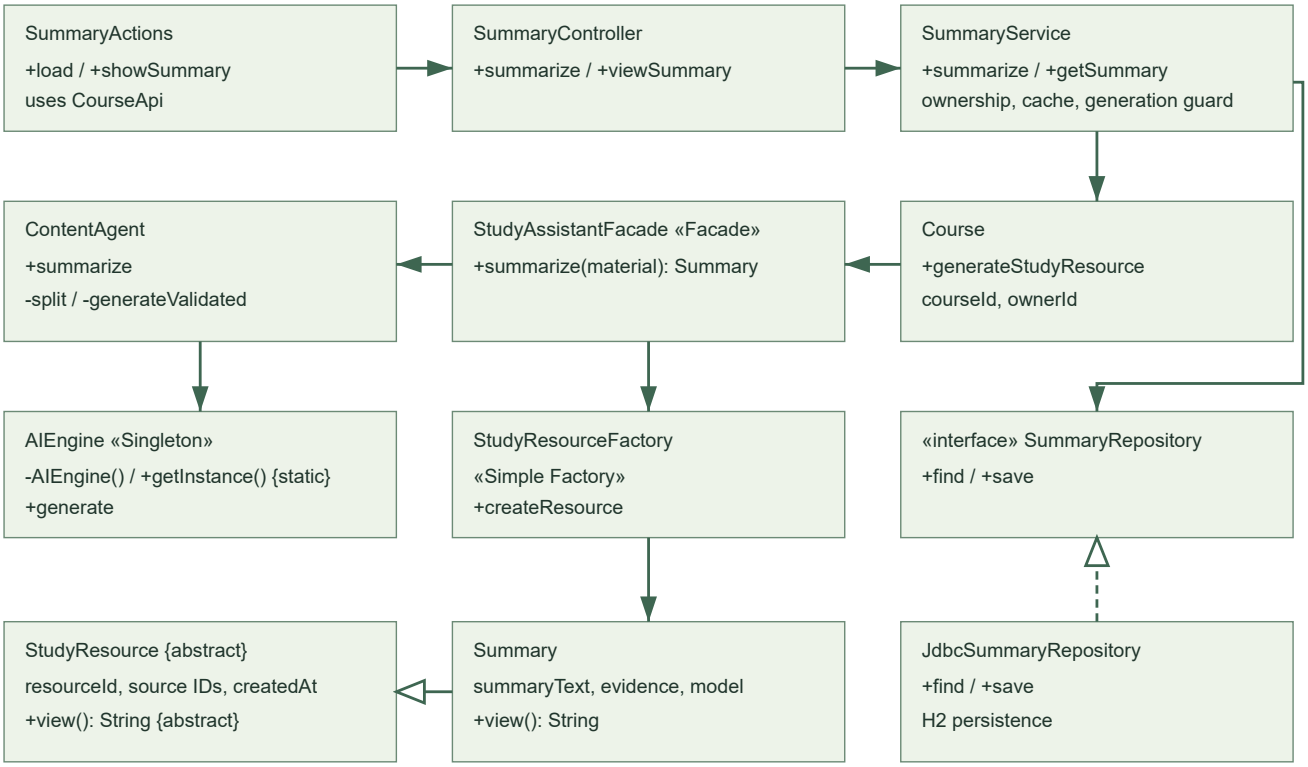
Stage 1 pattern	Implementation and purpose
Facade	`StudyAssistantFacade.summarize/extractConcepts/answerQuestion/generateStudyPlan/generateFlashcards/generateQuiz` coordinates the agent and resource factory, hiding model orchestration and construction from course/UI callers.
Singleton	`AIEngine` has a private constructor and holder-based `getInstance`. Agents share one HTTP connection pool and concurrency limit; provider settings remain immutable per call.

Factory	`StudyResourceFactory.createResource` centralizes resource identity, timestamp, source association, and provenance. This simple factory constructs `Summary`, `ConceptSet`, `StudyPlan`, `FlashcardDeck`, and `Quiz` from validated agent data; all five planned resource products are available.
Observer	`ScheduleSubject` defines attach/detach/notify; `StudyPlan` is the concrete subject. `ScheduleObserver` defines update; `Calendar` is the concrete observer. `notifyObservers` sends the changed plan to `Calendar.update`, keeping saved study events synchronized without StudyPlan containing calendar SQL. `PlanPersistence` makes both writes atomic. This follows the Stage 1 role assignment; the update method now receives the plan. Course/Quiz subjects remain pending.
Strategy	`GradingStrategy` defines a common grade operation. `DeterministicGradingStrategy` and `AIGradingStrategy` implement it. `QuizGradingService` is the context; `setStrategy` switches the active strategy per question. A fresh prototype context per grading operation prevents concurrent requests sharing a mutable strategy field. Singleton AIEngine is reused.

All five planned patterns are implemented. `CourseApiFactory` and Spring bean scopes are not counted as additional patterns.

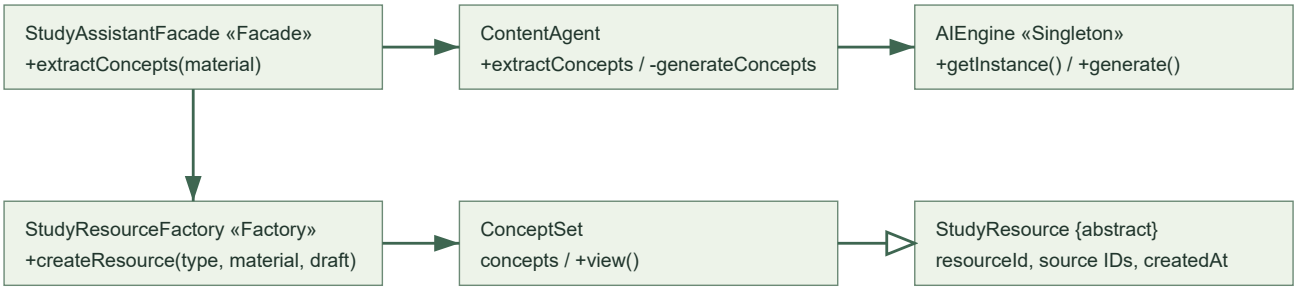
F01 adds a Repository abstraction: `CourseService` depends on `CourseRepository`, implemented by `JdbcCourseRepository`. This keeps SQL separate from application logic; it does not replace a planned pattern.

F03 UML class relationships; F01/F02 diagrams are linked below:

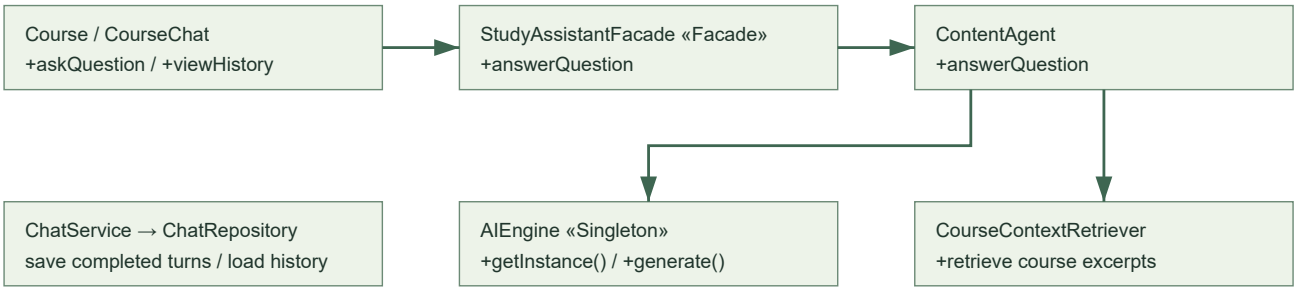


Updated UML/sequence diagrams: `documentation/diagrams/stage2-f01.md`, `stage2-f02.md`, `stage2-f03.md`, `stage2-f04.md`, `stage2-f05.md`, `stage2-f06.md`, `stage2-f07.md`, `stage2-f08.md`, and `stage2-f09.md`. They show the implemented classes and pattern roles; original Stage 1 diagrams are preserved.

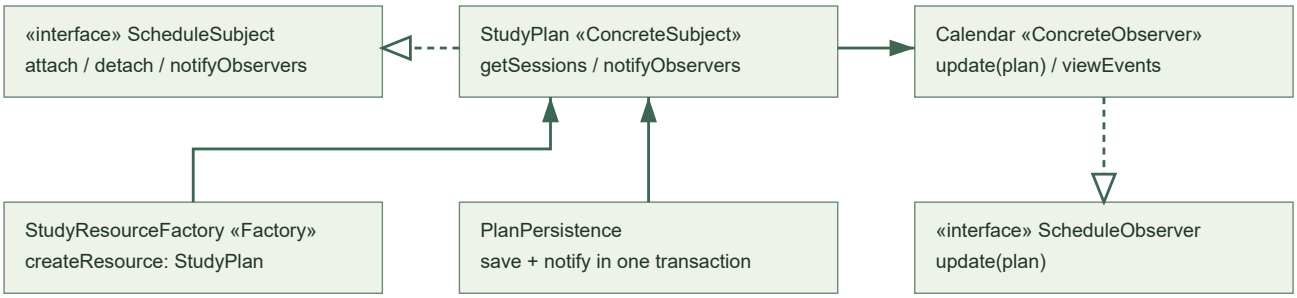
F04 adds a second product to the same pattern structure (full class and sequence details: `stage2-f04.md`):



F05 extends the same facade and singleton with course chat. Full class and sequence diagrams: `documentation/diagrams/stage2-f05.md`.



F06 adds StudyPlan as an Observer subject and a third factory product. Full class/sequence diagrams: `stage2-f06.md`.



F07 class and sequence diagrams: stage2-f07.md.

4. Changes and rationale

Stage 1 design	Implementation change	Reason and impact
UI → CourseController → User → Course (F01/UC-01/SD01).	Added CourseApi, CourseService, CourseRepository, JDBC, and H2.	Connects the separate apps and saves courses across restarts. The creation sequence is updated.
User owns a collection of courses.	User creates courses; repository queries retrieve them by owner UUID derived from username.	Supports persistent ownership. Changing the username changes the owner identity; account persistence is future work.
Authenticated student required; authentication mechanism unspecified.	One configured account, Spring Security, HTTP Basic, and CSRF protection.	Enforces the precondition. Credentials stay in ignored local configuration. Remote deployment requires HTTPS.
Four course fields; detailed limits unspecified.	Required fields, text limits, trimming, and ordered dates.	Rejects invalid courses consistently. Editing/deletion remain pending.
GUI and CLI required.	Java CLI calls the same REST API as the GUI.	Reuses validation and storage. Future agent features still need CLI support.

F02/UC-02/SD01: CourseController → Course.addMaterial → LearningMaterial; file rules unspecified.	Added course navigation, MaterialService, parser, repository, text previews, and browser downloads of originals. PDF/DOCX/TXT, size and extraction limits apply.	Students can read extracted text or save the complete document, preserving formatting and images. Durable BLOB/CLOB storage keeps originals and text consistent. No AI or OCR is claimed.
F03/UC-03/SD02: Facade → ContentAgent → shared AIEngine → resource Factory → Summary.	Agent returns validated draft data for factory construction; section planning, synthesis, indexed source evidence, cache/source hashes, and explicit regeneration were added.	Handles longer documents and malformed model output with bounded work. SummaryService adds ownership and persistent memory. These additions affect the F03 sequence; original pattern roles are retained.
F04/UC-04/SD02: use the facade, agent, AI engine, and factory to create and display a ConceptSet. Duplicate concepts or invalid AI output cause an error.	Kept the original classes and patterns. Added processing for long documents, saved concepts, viewing saved results, and regeneration. Matching concept names are merged instead of causing an error. Invalid AI output gets one correction attempt before an error is shown.	Users can reuse saved concepts and process longer documents. The agent checks the AI response before the factory creates the ConceptSet. If generation fails, previously saved concepts are kept. These additions support the original F04 feature.
F05/UC-05/SD02: CourseChat asks the facade and agent for an answer from course materials and saves the conversation.	Kept the original classes and methods. Added a search for relevant passages, course overviews, source quotations, recent conversation context, history pages, and one correction attempt for invalid or unfinished AI output. Answers return text plus source information instead of a plain String.	Helps users check answers and ask follow-ups. ChatService and the repository save complete exchanges and enforce course ownership. Word matching may miss relevant material, and the model can still make mistakes. No new design pattern is claimed for F05.
F06/UC-06/UC-11/SD03: AI chooses study topics and activities, Java schedules sessions, and StudyPlan notifies Calendar.	Implemented that flow with saved weekly windows, a shared minute budget, time zones, blocking events, and latest-plan storage. Calendar.update receives the changed plan. The factory assembles sessions from Java-selected times. Topic discovery now examines separate source passages; revisits use different practice and review tasks instead of repeating identical instructions. Course-wide plans carry citations instead of one material ID. Manual assessment notes remain available alongside F09 saved grades.	Makes availability and deadlines concrete and prevents conflicts across courses. The same plan/calendar transaction avoids partial updates. Facade, Singleton, Factory, and Observer keep their planned roles. Full F10 calendar views remain future work; F09 now supplies quiz results.
F07/UC-07/SD04: choose a card limit and source sections, generate a FlashcardDeck, and reveal answers on click.	Implemented the SD04 classes and methods. Added selected materials, an optional topic, validated source quotations, small generation batches, duplicate-question removal, and saved decks. Short batches continue with unused facts. Invalid cards are skipped while validated cards are kept, so one bad response no longer discards a useful deck. The limit is a maximum; fewer cards are allowed when sources support fewer distinct questions.	UC-07's main scenario repeats study-plan steps, so F07's feature description, Task 4, UML, and SD04 guide this implementation. Selected uploaded files represent sections; a separately uploaded section can be selected. No section-management system was added. Facade, Singleton, and Factory retain their planned roles.
F08/UC-08/SD04: generate a quiz from chosen topics, count, difficulty, and preferred types, then allow an attempt.	Kept the planned classes and methods. Added typed questions, saved quizzes, resumable versioned attempts, hidden draft answer keys, and post-finish source review. Questions use source-based blanks: AI selects the removed phrase and distractors; Java constructs the stem and checks the key against the source.	Initial model review found natural-language questions paired with unrelated keys. Constructing blanks from verified spans prevents that mismatch. Topic/type/difficulty selection remains AI-assisted; distractors still need review. F08 provides generation and attempts; F09 now adds scoring, feedback, and Strategy-based grading.
F09/UC-09/SD05: Quiz submits answers, the grading service selects a strategy, and GradeResult combines score, feedback, and weak areas.	Implemented the named classes/methods with per-question strategies, stored keys/points, a fixed semantic rubric, and weighted totals. QuizAttempt preserves retakes; a prototype grading context isolates strategy selection. Attempt and grade commit together. Cached results and grading older finished attempts are supported; saved weaknesses feed F06.	Per-question strategy calls support mixed quizzes. Atomic storage avoids a finished attempt without its grade. Live review exposed half-credit bias, so the model now chooses a categorical verdict with clearer examples; Java assigns points. Case/punctuation-only matches are deterministic. Model-written feedback also invented an answer, so Java now builds feedback from saved criteria. AI judgment remains a limitation and is labeled in the UI.

Stage 1 proposed ChatGPT, LangChain/RAG, and shared AIEngine.	Direct Java HTTP orchestration; holder-based singleton accepts provider settings per call. Local Ollama is selected.	Keeps the existing Java apps and enables local inference. F05 now retrieves course passages using word matching before generation. No LangChain, embeddings, or vector service is used.

No original feature or planned pattern has been removed or replaced.

5. AI tools and collaboration

Development tool: OpenAI Codex, a GPT-6-based coding agent; exact runtime model ID unavailable. Used for F01–F09 code, tests, diagrams, and reporting. **Application LLM:** local Ollama with qwen2.5:1.5b; no hosted API billing.

Interaction: the student requested F01/F02, a shorter report, then F03 with local Ollama and F04–F09 following the original UML. Codex implemented and checked the course, material, summary, concept, chat, study-planning, flashcard, quiz, and grading flows.

Tokens and cost: Codex totals remain unavailable; add platform usage/billing before submission. Successful summaries, concept sets, chat exchanges, study plans, flashcard decks, quizzes, and grades retain provider-reported token counts; -1 means unavailable. Local compute cost is not measured. Human review remains required.

6. Development and verification

One hundred and forty backend tests and the production frontend build pass. Coverage includes F01/F02 regression, F03 ownership/CSRF, section planning, output repair, quotation checks, cache/regeneration, timeouts, provider errors, and concurrent generation. Scripted-provider Chrome/CLI checks pass for summary progress, dialogs, saved results, regeneration, errors, mobile layout, and persistence. These fixtures test integration, not model quality. Real qwen2.5:1.5b verification also passed: CLI generation, browser viewing/regeneration, and a two-section document synthesized in three model calls. Source quotations were checked and the sample output reviewed. This verifies a working local integration, not general model accuracy.

Checks: `backend/mvnw.cmd clean verify`, frontend `mvnw.cmd package -Pproduction`, and `scripts/verify-f01.ps1`. Runtime checks use isolated test data and stop their own servers. A missing BCrypt encoding prefix was found and corrected during testing.

F02 runtime checks use `scripts/verify-f01.ps1 -IncludeF02`. Real-client checks caught a text-preview Accept-header mismatch and a parallel-upload CSRF session race. Correct response headers and serialized token/write requests fixed them.

Live testing exposed paraphrased quotations and missing JSON fields in the small model; constrained output and indexed evidence corrected these failures. F03 checks: `scripts/verify-f01.ps1 -IncludeF03` uses a test-only provider; `scripts/verify-f03-live.ps1` uses real Ollama and a safe lecture sample. No production fake-response fallback exists.

F04 adds thirteen backend checks, including paraphrased names, full excerpt coverage, dense lists, repair feedback, ownership, cache/regeneration, and provider failures. `verify-f01.ps1 -IncludeF04` covers GUI/CLI and restart persistence; `verify-f03-live.ps1 -IncludeF04` passed real Ollama software-design/biology extraction, browser regeneration, and multi-section consolidation. A reported failure exposed literal-name matching against 12 sampled excerpts. Removing that rule and expanding coverage fixes false rejections while retaining source-ID checks. The small model can still produce broad topics, semantic duplicates, inaccurate explanations, or irrelevant citations; exact quotations establish provenance, not factual accuracy.

F05 adds eighteen backend checks for retrieval, ownership, citations, history, failures, overviews, and unfinished answers. GUI/CLI checks cover source viewing and persistence after restart. Live Ollama checks cover questions, follow-ups, unrelated topics, and overviews. Testing corrected a missing response schema, overly strict keyword checks, and earlier refusals affecting new topics. A reported answer stopped after “follow these steps:”; the agent now rejects dangling introductions and empty list items and requests one complete replacement. Tests verify that failed correction preserves history and complete short answers remain valid. Real PDF/CLI/browser verification (`verify-f05-completeness.ps1`) returned substantive guidance for the reported

Stage 2 question. A separate layout regression reproduced long answers running beyond the right edge when custom wrapping was missing. Chat answers now enforce wrapping and width limits directly. Browser checks compare the full saved answer and its final sentence at 1920, 1280, and 390 pixels, including without the custom text class. These checks do not guarantee factual accuracy or detect every incomplete answer.

F06 adds twenty-two backend checks for profiles, availability, weekly budgets across courses, busy times, deadlines, daylight-saving durations, ownership, provider errors, concurrency, and atomic Observer rollback. `verify-f01.ps1 -IncludeF06` passes GUI/CLI editing, regeneration, insufficient-time errors, mobile layout, and profile/plan/calendar persistence after restart with the provider stopped. Real Ollama (`verify-f06-live.ps1`) generated and regenerated a four-session biology plan around a work block and quiz deadline. Quotes were checked and recommendations reviewed. A reported two-topic cycle exposed one broad AI response being repeated unchanged. Passage-by-passage topic discovery, different revisit activities, and complete-sentence checks address it. Four additional regression tests cover topic diversity, merging common paraphrased comparison titles, different repeat tasks, and failed repair preserving the saved plan/calendar. Final live Ollama verification produced three biology priorities across four sessions with different reading, practice, and review work; browser regeneration and calendar synchronization passed. The model still needs human review; deterministic scheduling checks are independent of model quality.

F07 has twenty-six backend checks covering ownership, selection, counts up to 20, repair, duplicates, source evidence, history, and recovery from invalid output. Short batches continue toward the maximum using unused facts; invalid multi-card responses fall back to single-card requests. Mixed batches retain valid cards, repeatedly unusable facts are skipped, and later validation failures can save a smaller validated deck with an explanation. No valid cards means no new deck, and old decks are preserved. PDF line wrapping is joined into complete source sentences; unfinished headings are excluded. GUI/CLI checks cover reveal/navigation, saved reopening, failures, mobile layout, and restart retrieval without a provider. A forced late failure also verifies the smaller-deck notice and preservation of earlier decks. Real Ollama generated 10/10 biology cards in the earlier count check. The assignment-PDF recovery check returned 9/10 validated cards and passed exact quotation, saved-viewing, reveal/navigation, and mobile checks; a failed card no longer discards the useful deck. Questions may still be vague, overlapping, or misleading despite exact source answers, so source review remains necessary.

F08 adds eighteen backend checks for counts/types/difficulty, selected materials, ownership/CSRF, answer-key hiding, draft/resume, stale versions, completed-attempt immutability, invalid submissions, repair, provider failures, duplicate output, and history pages. The GUI/CLI checks cover generation, mixed questions, save/resume/finish, failed generation preserving data, mobile layout, and restart retrieval with the model stopped. Initial live review exposed unrelated question/key pairs; source-based blanks now require that restoring the key reproduces the cited statement. Real qwen2.5:1.5b generated four mixed questions with matching keys, and browser draft/resume/finish and exact quotations passed. A later ten-question failure exposed the three-failed-facts cutoff and a global ban on repeated answer terms. The generator now offers prevalidated source phrases, continues to other facts within its budget, and allows a term in different question contexts. Regression tests cover both recovery cases. Real qwen2.5:1.5b subsequently generated all ten mixed questions and passed browser save/resume/finish, source checks, and mobile layout. Distractor ambiguity and semantic difficulty still need human review.

F09 adds eighteen backend checks for weighted marks, partial credit, binary choices, exact/case matching without AI, criteria, ownership/CSRF, repair, provider failures, cached/legacy grades, concurrent submissions, atomic rollback, retakes, and planning integration. CLI JSON output now uses UTF-8 so source text survives Windows PowerShell round trips. Scripted GUI/CLI checks cover score/feedback display, preserved drafts after grading failure, retry, mobile layout, and cached results after restart without a provider. Real qwen2.5:1.5b grading was checked per question: 1, 1, 0, 0 points, including a correct phrase in a sentence and an unrelated answer, rather than only checking the 2/4 total. AI judgments remain approximate and need source review.

Toronto reporting period	Minimum code commits	Observed local evidence
October 20–26, 2026	5	No qualifying local commits found; remote evidence needs checking.
October 27–November 2, 2026	5	a0db921: starter; 42853d5: F01; 8ab568b: F02. b26894b: F03. b0c6578: F04. Five local code commits observed; push status not verified.

November 3–9, 2026	5	67ca915: F06 (including F05), November 7. 48543b6: F07 and F06 fixes, November 8. dfb3e51: F08, November 8. Three local code commits observed; push status not verified. F09 is uncommitted.

Local credentials and database files are ignored by Git. `backend/config.local.properties.example` provides placeholders; setup instructions are in the README. F01 requires no API key.

Before November 10: complete F10, full-system diagrams, videos, cost records, and commit evidence. Check additional baseline instructions; only the supplied Stage 1 report and Stage 2 instructions were reviewed.

7. Demonstration index

Feature	Video status	Demonstration scope
F01 Course Creation	Recording and accessible link/timestamp pending.	Identify F01; show GUI and CLI creation, input errors, dashboard output, and persistence. Explain that F01 is deterministic. Keep credentials hidden.
F02 Upload Learning Materials	Recording and link/timestamp pending.	Show course navigation, GUI/CLI upload, saved files, preview, original downloads, invalid/type/size errors, and persistence. Identify deterministic extraction and pending AI processing.
F03 Document Summarization	Recording/link pending.	Show real Ollama GUI/CLI generation, source quotations, saved summaries, regeneration, and error cases. Explain section planning and output validation.
F04 Concept Extraction	Recording/link pending.	Show real GUI/CLI extraction, concept names/details and evidence, saved viewing, regeneration, input/provider errors, and section consolidation.
F05 Course Questions	Recording/link pending.	Show GUI/CLI questions, follow-ups, source viewing, unsupported questions, errors, and saved history after restart.
F06 Personalized Study Plans / UC-11	Recording/link pending.	Show profile editing, busy/quiz events, real AI recommendations, conflict-free sessions, calendar sync, regeneration errors, and saved data after restart through GUI/CLI.
F07 Flashcard Generation	Recording/link pending.	Show GUI/CLI material selection, real model generation, answer reveal, navigation, quotations, errors, and saved decks after restart.
F08 Quiz Generation	Recording/link pending.	Show GUI/CLI generation, types/difficulty, source-based blanks, hidden keys, draft/resume, finish/review, errors, and persistence. Identify F08 generation and F09 grading.
F09 Automatic Quiz Grading	Recording/link pending.	Show mixed scoring, partial credit, feedback/weak areas, grading failure with preserved drafts, cached results, old-attempt grading, and next-plan personalization through GUI/CLI. Identify AI judgment limits.
F10 Calendar	Partial; video pending.	F06 demonstrates event inputs and synchronized study sessions. Complete the full calendar feature later.

Automated browser checks do not replace demonstration videos. Update this source and regenerate `stage2-report.pdf` as features are completed.